Q2. Hotel Billing System (Multiple Inheritance using One Class and One Interface)

Score: 10.00 **TC: 3/3 passed**

CODE

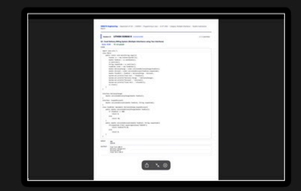
```
import java.util.*;

class Main{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        int roomNumber=sc.nextInt();
        sc.nextLine();
        String guestName = sc.nextLine();
        double roomRent = sc.nextDouble();
        HotelBill b = new HotelBill();
        b.setRoomDetails(roomNumber, guestName, roomRent);
        b.displayDetails();
        sc.close();
    }
}
class Room{
    int roomNumber;
    String guestName;
    double roomRent;
    void setRoomDetails(int roomNumber, String guestName, double roomRent){
        this.roomNumber = roomNumber;
        this.guestName = guestName;
        this.roomRent = roomRent;
    }
}

interface ServiceCharge{
    double calculateServiceCharge();
}
class HotelBill extends Room implements ServiceCharge{
    public double calculateServiceCharge(){
        if(roomRent >= 5000)
            return roomRent * 0.12;
        else
            return roomRent * 0.08;
    }
    void displayDetails(){
        double serviceCharge = calculateServiceCharge();
        double finalBill = roomRent + serviceCharge;

        System.out.println("Room Number : "+roomNumber);
        System.out.println("Guest Name : "+guestName);
        System.out.println("Room Rent : "+roomRent);
        System.out.println("Service Charge : "+serviceCharge);
        System.out.println("Final Bill : "+finalBill);
    }
}
```

INPUT

```
205
Ananya
6000
```

OUTPUT

```
Room Number : 205
Guest Name : Ananya
Room Rent : 6000.0
Service Charge : 720.0
Final Bill : 6720.0
```



Student 23 **LITHISH KUMAR R** 192424169RD 2 / 2 submitted

Q1. Food Delivery Billing System (Multiple Inheritance using Two Interfaces)

Score: 10.00 **TC: 3/3 passed**

CODE

```
import java.util.*;
class Main{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        double foodCost = sc.nextDouble();
        sc.nextLine();
        String couponCode = sc.nextLine();
        FoodOrder order = new FoodOrder();
        double deliveryCharge = order.calculateDeliveryCharge(foodCost);
        double discount = order.calculateDiscount(foodCost,couponCode);
        double finalBill = foodCost + deliveryCharge - discount;
        System.out.println("Food Cost : "+foodCost);
        System.out.println("Delivery Charge : "+deliveryCharge);
        System.out.println("Discount : "+discount);
        System.out.println("Final Bill : "+finalBill);
        sc.close();
    }
}

interface DeliveryCharge{
    double calculateDeliveryCharge(double foodCost);
}
interface  CouponDiscount{
    double calculateDiscount(double foodCost, String couponCode);
}
class FoodOrder implements DeliveryCharge,CouponDiscount{
    public double calculateDeliveryCharge(double foodCost){
        if (foodCost >= 500)
            return 0;
        else
            return 50;
    }
    public double calculateDiscount(double foodCost, String couponCode){
        if(couponCode.trim().equalsIgnoreCase("SAVE10"))
            return foodCost*0.10;
        else
            return 0;
    }
}
```

INPUT

```
600
SAVE10
```

OUTPUT

```
Food Cost:600.0
Delivery Charge:0.0
Discount:60.0
Final Bill:540.0
```